What is this sheet?

This lesson is a collection of various things we take for granted in the unit. You can use it as a reference for study, as the results of these is assess-able (but the proofs aren't).

1+2+3+...

Result

$$1 + 2 + \cdots + (n-1) + n \;=\; \frac{n\,(n+1)}{2} = O\left(n^2\right)$$

Explanation

Extremely often in complexity analysis, we come across this beast of a sum

$$1 \;+\; 2 \;+\; 3 \;+\; \cdots + \;(n-1) \;+\; n,$$

or something that looks just like it. Your knee jerk reaction might be to claim that this is $O(n)$, by looking at the dominating terms, but by that same logic, we could say

$$n \;=\; 1 + 1 + \cdots + 1 \;=\; O\,(1)$$

Which is evidently preposterous. So we should try getting a closed form for this. There are multiple possible proofs that give us what we want:

Proof #1: Direct

We'll start by writing out our sum again:

$$1 \;+\; 2 \;+\; 3 \;+\; \cdots + \;(n-1) \;+\; n,$$

and again, but this time in reverse:

$$n \;+\; (n-1) \;+\; \cdots + \;3 \;+\; 2 \;+\; 1$$

Look at what happens when we add these two sequence together, element by element:

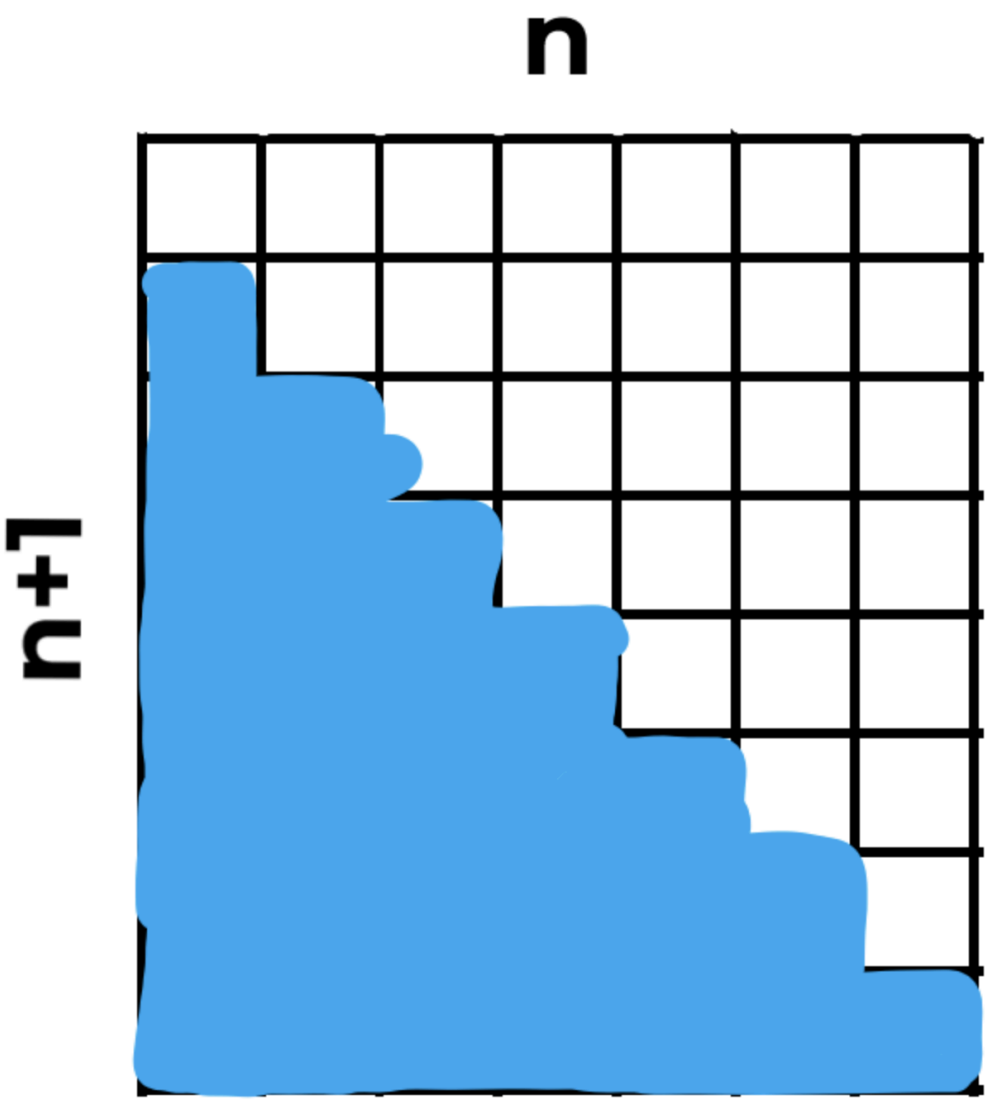
$$\begin{aligned}
&(1+n) \;+\; (2 \;+\; (n-1)) \;+\; (3 \;+\; (n-2)) \;+\; \cdots + \;((n \;-\; 1) \;+\; 2) \;+\; (n \;+\; 1) \\
&= (n+1) \;+\; (n+1) \;+\; \cdots + \;(n+1) \\
&= n\,(n+1)\,.
\end{aligned}$$

So we know that two copies of our sequence is equal to $n(n+1)$. So we then must have that

$$1 + 2 + \cdots + (n-1) + n \;=\; \frac{n\,(n+1)}{2} = O\left(n^2\right)$$

Proof #2: Graphical

Consider a grid-rectangle of size $n \times (n+1)$, and colour the bottom 'triangle' of this grid black:



Two results arise:

1. What is the area of the entire rectangle?

▼ Expand

$n \times (n+1)$

2. What is the area of the blue triangle?

▼ Expand

$1 + 2 + 3 + \cdots + n - 1 + n$

With these two results in hand, we can come to a similar conclusion as proof 1.

Proof #3: Extensible

While the previous two proofs were useful, they didn't help us with a more general problem, of finding a closed form for the following:

$$1^p + 2^p + 3^p + \cdots + (n-1)^p + n^p,$$

Which this final proof may help with.

Start by considering the following:

$$k^2 - (k-1)^2 \;=\; 2k \;-\; 1.$$

From this we can gleam that

$$k \;=\; \frac{k^2 - (k-1)^2 \;+\; 1}{2}$$

This formula, however excessive, can be applied to our original sequence:

$$1 + 2 + 3 + \cdots + (n-1) + n$$

$$= \frac{1^2 - 0^2 + 1}{2} + \frac{2^2 - 1^2 + 1}{2} + \frac{3^2 - 2^2 + 1}{2} + \cdots + \frac{(n-1)^2 - (n-2)^2 + 1}{2} + \frac{(n)^2 - (n-1)^2 + 1}{2}$$

I know this looks complicated, but we can cancel out many of these terms (notice the 1^2 is cancelled by the first 2 terms, the 2^2 is cancelled by the 2nd and 3rd terms, and so on:

$$= \frac{-0^2 + 1}{2} + \frac{1}{2} + \frac{1}{2} + \cdots + \frac{1}{2} + \frac{n^2 + 1}{2}$$

$$= n \times \frac{1}{2} + \frac{n^2}{2} = \frac{n^2 + n}{2} = \frac{n\,(n+1)}{2}.$$

Extra Tasks

1. Find a closed form for $2 + 4 + 6 + \cdots + (2n-2) + 2n$.

2. Find a closed form for $1 + 3 + 5 + \cdots + (2n-1) + (2n+1)$.

3. Find a closed form for $1^2 + 2^2 + 3^2 + \cdots + (n-1)^2 + n^2$. What is the big O notation for this?

▼ Expand

Hint: Use Proof #3 for question 3.

1 + 1/2 + 1/4 + 1/8 + ...

Another useful sum is 1 + 1/2 + 1/4 + 1/8, where each term is half as big as the last.

A simple way to realise that this is bounded above by 2 always, is to note the following:

$$S = 1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{2^i}$$

$$2S = 2 + 1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{2^{i-1}}$$

$$2S - S = 2 - \frac{1}{2^i}$$

$$S \leq 2$$

That second to last formula tells us that as we increase i, our sum moves closer and closer to 2, only ever being $\frac{1}{2^i}$ distance away.

Why does repeated halvings yield logarithmic complexity?

Example

A very common pattern in code is a reduction of a variable by halving, or reduction in search space.

<> Python

```python
def print_some_elements(my_list):
    left = 0
    right = len(my_list)-1
    while right > 0:
        print(my_list[right])
        right //= 2

if __name__ == "__main__":
    print_some_elements("ABCDEFGHIJKLMNOPQRSTUVWXYZ")
```

In these particular cases, the number of iterations is $O(\log(n))$, where n is the size of your collection.

Proof

What is helpful in this example (and many other complexity proofs), is to try parameterize the current value in terms of:

1. The value at the beginning

2. The iteration we are on (call it i)

As an example, in the above code, we might make an expression for the value of `right` as time goes on:

0. In iteration 0, `right = original_right`.

1. In iteration 1, `right = original_right / 2` (about)

2. In iteration 2, `right = original_right / 2 / 2 = original_right / 4` (about)

...

From this, we can find that the closed form for this is:

$$\text{right} = \frac{\text{original right}}{2^i}$$

Which begs the question, at what iteration does `right > 0` equal false? Well this occurs when `right = 0`, or in other words the previous iteration must have `right = 1`. Let's solve for `i` to see aroundabout which iteration gives us this value:

$$1 = \frac{\text{original right}}{2^i}$$

$$2^i = \text{original right}$$

$$\log_2\left(2^i\right) = \log_2\left(\text{original right}\right)$$

$$i = \log_2\left(\text{original right}\right)$$

By using some high school log laws.

Extended Tasks

1. What is the complexity of the following function:

<> Python

```python
def print_some_elements(my_list):
    left = 0
    right = len(my_list)-1
    while right > 0:
        print(my_list[left:right])
        right //= 2

if __name__ == "__main__":
    print_some_elements("ABCDEFGHIJKLMNOPQRSTUVWXYZ")
```

Hint:

▼ Expand

This is a trick question

2. Why does the base of a logarithm not matter for complexity?

Hint:

▼ Expand

Try to come up with a formula that compares $\log_2(n)$ to $\log_{10}(n)$